06 — Selección de Características

Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

Este notebook selecciona las características usando **todas las imágenes** (8274 tras eliminar duplicados exactos). **No hay hold-out**: la evaluación se hace con validación cruzada sobre las 8274 y la selección de características se ajusta **dentro de cada fold**.

Objetivo

Elegir el método de selección (Chi^2, Mutual Information o importancia de Random Forest) y el número de características `N`, maximizando el F1-macro en validación cruzada.

Protocolo

1. Carga y deduplicación exacta de las 8400 imágenes → 8274.
2. **CV 5-fold estratificada sobre las 8274**.
3. En cada fold, el selector se ajusta **solo con el fold de entrenamiento**.
4. Regla de **1 error estándar** para elegir el menor `N` dentro de 1 SE del mejor.
5. Persistencia de `X_full.npy`, `y_full.npy`, la selección y los scores.

Flujo

```
features_tbx11k.csv (8400 x 3300)
        |
        v
 Deduplicacion exacta -> 8274 imagenes
        |
        v
 CV 5-fold: selector ajustado dentro de cada fold
 (Chi2 / MI / RF) x N en {100..3300}
        |
        v
 Regla 1-SE (error estandar) -> metodo + N
        |
        v
 X_full.npy / y_full.npy + seleccion
        |
        v
    Notebook 07
  Desbalanceo (CV sobre 8274)
```

0. Configuración del entorno

In [1]:
```python
# Se configura el path del proyecto y se importan las librerías necesarias.
# Referencias:
#   validación cruzada: https://scikit-learn.org/stable/modules/cross_validation.html
#   selección de características: https://scikit-learn.org/stable/modules/feature_selection.html

import sys, os, warnings, json, time, hashlib
import cv2
warnings.filterwarnings('ignore')

from pathlib import Path
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score

PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

from src.selection import feature_scores
from src.visualization import set_publication_style

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

DATA_DIR = Path('..') / 'data'

set_publication_style()
print('Entorno configurado correctamente.')
```

```
Entorno configurado correctamente.
```

1. Carga y deduplicación exacta

In [2]:

```
# Se cargan todas las imágenes y se eliminan duplicados exactos de features.
# No se crea ningún split train/test: se usan las 8274 imágenes para CV.
# Referencia: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.drop_duplicates.html

CSV_PATH = DATA_DIR / 'features_tbx11k.csv'
if not CSV_PATH.exists():
    raise FileNotFoundError(
        f'No se encontró {CSV_PATH.resolve()}. Ejecuta primero el notebook 05.'
    )

df = pd.read_csv(CSV_PATH)
META_COLS = ['image_path', 'class_name', 'label']
feat_cols = [c for c in df.columns if c not in META_COLS]

n_antes = len(df)

# Verificación: un mismo vector de features no debe tener etiquetas distintas.
claves = pd.util.hash_pandas_object(df[feat_cols], index=False)
conflictos = (
    df.assign(_clave=claves)
      .groupby('_clave')['label']
      .nunique()
      .gt(1)
      .sum()
)
print(f'Duplicados con etiquetas en conflicto: {conflictos}')

df = df.drop_duplicates(subset=feat_cols, keep='first').reset_index(drop=True)
print(f'Duplicados exactos eliminados: {n_antes - len(df)}')
print(f'Imágenes tras deduplicación  : {len(df)}')
print(df['class_name'].value_counts().to_string())

# Verificación adicional: duplicados exactos a nivel de archivo de imagen.
def _md5(path):
    h = hashlib.md5()
    with open(path, 'rb') as f:
        for chunk in iter(lambda: f.read(1 << 20), b''):
            h.update(chunk)
    return h.hexdigest()

if df['image_path'].map(os.path.exists).all():
    hashes_img = df['image_path'].map(_md5)
    n_img_dup = int(hashes_img.duplicated().sum())
    print(f'Duplicados exactos a nivel de imagen: {n_img_dup}')
```

```
    if n_img_dup:
        tmp = df.assign(_img_hash=hashes_img)
        conflictos_img = int(tmp.groupby('_img_hash')['label'].nunique().gt(1).sum())
        print(f'Conflictos de etiqueta a nivel de imagen: {conflictos_img}')
        df = (tmp.drop_duplicates('_img_hash', keep='first')
                 .drop(columns='_img_hash')
                 .reset_index(drop=True))
        print(f'Imágenes tras dedup por hash: {len(df)}')
else:
    print('Aviso: algunas rutas no existen; no se verificó el hash de imagen.')

# Verificación perceptual: dHash 8x8 (detecta duplicados visuales exactos,
# no recortes/rotaciones).
def _dhash(path):
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        return None
    img = cv2.resize(img, (9, 8), interpolation=cv2.INTER_AREA)
    diff = img[:, 1:] > img[:, :-1]
    return int(''.join('1' if x else '0' for x in diff.flatten()), 2)

if df['image_path'].map(os.path.exists).all():
    dhashes = df['image_path'].map(_dhash)
    print(f'Duplicados perceptuales exactos (dHash): {int(dhashes.duplicated().sum())}')

X = df[feat_cols].to_numpy(dtype=np.float32)
y = df['label'].to_numpy()
meta = df[META_COLS].copy()

np.save(DATA_DIR / 'X_full.npy', X)
np.save(DATA_DIR / 'y_full.npy', y)
meta.to_csv(DATA_DIR / 'full_meta.csv', index=False)

print()
print(f'X_full : {X.shape} ({X.dtype})')
print(f'y_full : {y.shape}')
print(f'Features: {len(feat_cols)}')
```

```
Duplicados con etiquetas en conflicto: 0
Duplicados exactos eliminados: 126
Imágenes tras deduplicación  : 8274
class_name
health    3800
sick      3674
tb         800
Duplicados exactos a nivel de imagen: 0
Duplicados perceptuales exactos (dHash): 1084

X_full : (8274, 3300) (float32)
y_full : (8274,)
Features: 3300
```

2. Redundancia HOG/PHOG (medida, no supuesta)

In [3]:

```
# Se mide la correlación entre una muestra de features HOG y PHOG.
# Es una medición sobre una muestra de filas y columnas, no una afirmación por tamaño.
# Referencia: https://numpy.org/doc/stable/reference/generated/numpy.corrcoef.html

hog_cols  = [c for c in feat_cols if c.startswith('hog_')]
phog_cols = [c for c in feat_cols if c.startswith('phog_')]

rng = np.random.RandomState(RANDOM_STATE)
n_rows = min(2000, len(df))
n_cols = 100

rows = rng.choice(len(df), size=n_rows, replace=False)
hog_sample  = rng.choice(len(hog_cols), size=min(n_cols, len(hog_cols)), replace=False)
phog_sample = rng.choice(len(phog_cols), size=min(n_cols, len(phog_cols)), replace=False)
cols = [hog_cols[i] for i in hog_sample] + [phog_cols[i] for i in phog_sample]

M = df.iloc[rows][cols].to_numpy(dtype=np.float32)
corr = np.corrcoef(M, rowvar=False)
```

```
h = len(hog_sample)

within_hog = np.abs(corr[:h, :h][np.triu_indices(h, 1)])
within_phog = np.abs(corr[h:, h:][np.triu_indices(len(phog_sample), 1)])
between = np.abs(corr[:h, h:])

redundancia = {
    'n_filas_muestra': int(n_rows),
    'n_hog_muestra': int(h),
    'n_phog_muestra': int(len(phog_sample)),
    'corr_abs_media_hog': float(np.nanmean(within_hog)),
    'corr_abs_media_phog': float(np.nanmean(within_phog)),
    'corr_abs_media_hog_phog': float(np.nanmean(between)),
}
print(json.dumps(redundancia, indent=2))
with open(DATA_DIR / '06_redundancia_hog_phog.json', 'w', encoding='utf-8') as f:
    json.dump(redundancia, f, ensure_ascii=False, indent=2)
```


```
{
  "n_filas_muestra": 2000,
  "n_hog_muestra": 100,
  "n_phog_muestra": 100,
  "corr_abs_media_hog": 0.10849828345440873,
  "corr_abs_media_phog": 0.10689951537207568,
  "corr_abs_media_hog_phog": 0.10440392640534342
}
```


3. Validación cruzada de la selección de características

In [4]:

```
# Para cada fold se calculan los scores de los tres métodos usando SOLO el train
# del fold; después se evalúa cada N con un LogisticRegression(class_weight='balanced').
# El selector nunca ve el fold de validación.
# Referencia: https://scikit-learn.org/stable/modules/feature_selection.html

N_VALUES = [100, 300, 500, 1000, 1500, 2000, 2500, 3300]
CV_FOLDS = 5
cv = StratifiedKFold(n_splits=CV_FOLDS, shuffle=True, random_state=RANDOM_STATE)

filas = []
rankings = {m: {} for m in ['Chi2', 'MI', 'RF']}
t0 = time.time()

for fold, (tr, va) in enumerate(cv.split(X, y), start=1):
    X_tr, X_va = X[tr], X[va]
    y_tr, y_va = y[tr], y[va]

    scores = {
        'Chi2': feature_scores('Chi2', X_tr, y_tr, RANDOM_STATE),
        'MI':   feature_scores('MI', X_tr, y_tr, RANDOM_STATE),
        'RF':   feature_scores('RF', X_tr, y_tr, RANDOM_STATE),
    }

    for method, sc in scores.items():
        orden = np.argsort(sc)[::-1]
        rankings[method][fold] = orden
        for n in N_VALUES:
            if n > len(orden):
                continue
            idx = orden[:n]
            scaler = StandardScaler()
            Xtr_s = scaler.fit_transform(X_tr[:, idx])
            Xva_s = scaler.transform(X_va[:, idx])
            clf = LogisticRegression(
                class_weight='balanced', max_iter=1000, random_state=RANDOM_STATE,
            )
            clf.fit(Xtr_s, y_tr)
            pred = clf.predict(Xva_s)
            filas.append({
                'method': method,
                'n': n,
                'fold': fold,
                'f1_macro': f1_score(y_va, pred, average='macro'),
```

```
            })

    print(f'fold {fold}/{CV_FOLDS} listo ({time.time() - t0:.1f}s)')

resultados_fold = pd.DataFrame(filas)
resumen = (
    resultados_fold
    .groupby(['method', 'n'])['f1_macro']
    .agg(['mean', 'std'])
    .reset_index()
)
resumen['se'] = resumen['std'] / np.sqrt(CV_FOLDS)
resumen = resumen.sort_values(['mean', 'n'], ascending=[False, True]).reset_index(drop=True)

print()
print('Resultados (media, DE y error estándar):')
print(resumen.round(6).to_string(index=False))

resumen.to_csv(DATA_DIR / '06_cv_resultados.csv', index=False)
resultados_fold.to_csv(DATA_DIR / '06_cv_por_fold.csv', index=False)
```

```
fold 1/5 listo (83.4s)
fold 2/5 listo (166.0s)
fold 3/5 listo (249.9s)
fold 4/5 listo (331.7s)
fold 5/5 listo (414.1s)

Resultados (media, DE y error estándar):
method    n     mean      std       se
    RF 2500 0.935983 0.006475 0.002896
    RF 1000 0.934914 0.004552 0.002036
    RF 2000 0.934309 0.006296 0.002816
    RF 1500 0.933234 0.003388 0.001515
    MI 2500 0.931121 0.007632 0.003413
    RF 3300 0.931042 0.007776 0.003477
  Chi2 3300 0.930952 0.007884 0.003526
    MI 3300 0.930952 0.007884 0.003526
  Chi2 2500 0.930774 0.003539 0.001583
    MI 1500 0.929530 0.008343 0.003731
    MI 2000 0.929061 0.011236 0.005025
    MI 1000 0.926846 0.009503 0.004250
  Chi2 2000 0.926024 0.006152 0.002751
  Chi2 1500 0.925604 0.006003 0.002685
    RF  500 0.925355 0.006441 0.002880
    RF  300 0.924594 0.007619 0.003407
  Chi2 1000 0.917788 0.006693 0.002993
    MI  500 0.914750 0.009435 0.004219
    MI  300 0.905453 0.006987 0.003125
    RF  100 0.897894 0.006096 0.002726
  Chi2  500 0.885862 0.006999 0.003130
    MI  100 0.879292 0.006285 0.002811
  Chi2  300 0.857127 0.014547 0.006506
  Chi2  100 0.809704 0.012620 0.005644
```

4. Selección por regla de 1 error estándar

In [5]:

```
# Regla de 1 error estándar (1-SE): se toma el mejor F1-macro medio y se aceptan
# configuraciones dentro de 1 error estándar (std / sqrt(k)) del mejor.
# Entre ellas se elige el menor N y, a igual N, el mayor F1.
# Referencia: Breiman et al. (1984), regla de 1-SE.

best = resumen.loc[resumen['mean'].idxmax()]
umbral = best['mean'] - best['se']

candidatos = resumen[resumen['mean'] >= umbral].copy()
candidatos = candidatos.sort_values(['n', 'mean'], ascending=[True, False])
sel = candidatos.iloc[0]

METHOD_SEL = sel['method']
N_SEL = int(sel['n'])

# Estabilidad de la selección entre folds (Jaccard del top-N).
```

```python
top_sets = [set(rankings[METHOD_SEL][f][:N_SEL]) for f in range(1, CV_FOLDS + 1)]
jaccard = []
for i in range(len(top_sets)):
    for j in range(i + 1, len(top_sets)):
        jaccard.append(len(top_sets[i] & top_sets[j]) / len(top_sets[i] | top_sets[j]))
estabilidad = float(np.mean(jaccard))
print(f'Estabilidad (Jaccard medio top-{N_SEL} entre folds): {estabilidad:.4f}')

print('Regla de 1 error estándar:')
print(f'  Mejor configuración : {best["method"]}, N={int(best["n"])} (F1={best["mean"]:.6f})')
print(f'  Error estándar      : {best["se"]:.6f}')
print(f'  Umbral              : {umbral:.6f}')
print()
print(f'Configuraciones candidatas: {len(candidatos)}')
print(candidatos.round(6).to_string(index=False))
print()
print(f'Seleccionada (menor N dentro de 1 SE): {METHOD_SEL}, N={N_SEL}')
print(f'  F1-macro CV : {sel["mean"]:.6f} +/- {sel["std"]:.6f}')

seleccion = {
    'criterio': '1-SE con error estandar',
    'n_folds': CV_FOLDS,
    'metodo': METHOD_SEL,
    'n_features': N_SEL,
    'mejor_metodo': best['method'],
    'mejor_n': int(best['n']),
    'mejor_f1': float(best['mean']),
    'error_estandar': float(best['se']),
    'umbral': float(umbral),
    'estabilidad_jaccard_media': estabilidad,
}
with open(DATA_DIR / '06_seleccion.json', 'w', encoding='utf-8') as f:
    json.dump(seleccion, f, ensure_ascii=False, indent=2)

# Scores definitivos sobre TODAS las imágenes (solo para guardar nombres/scores;
# el modelo final volverá a ajustar el selector dentro de su propio pipeline).
scores_all = feature_scores(METHOD_SEL, X, y, RANDOM_STATE)
orden_all = np.argsort(scores_all)[::-1]
nombres_sel = [feat_cols[i] for i in orden_all[:N_SEL]]
with open(DATA_DIR / '06_selected_feature_names.json', 'w', encoding='utf-8') as f:
    json.dump(nombres_sel, f, ensure_ascii=False, indent=2)

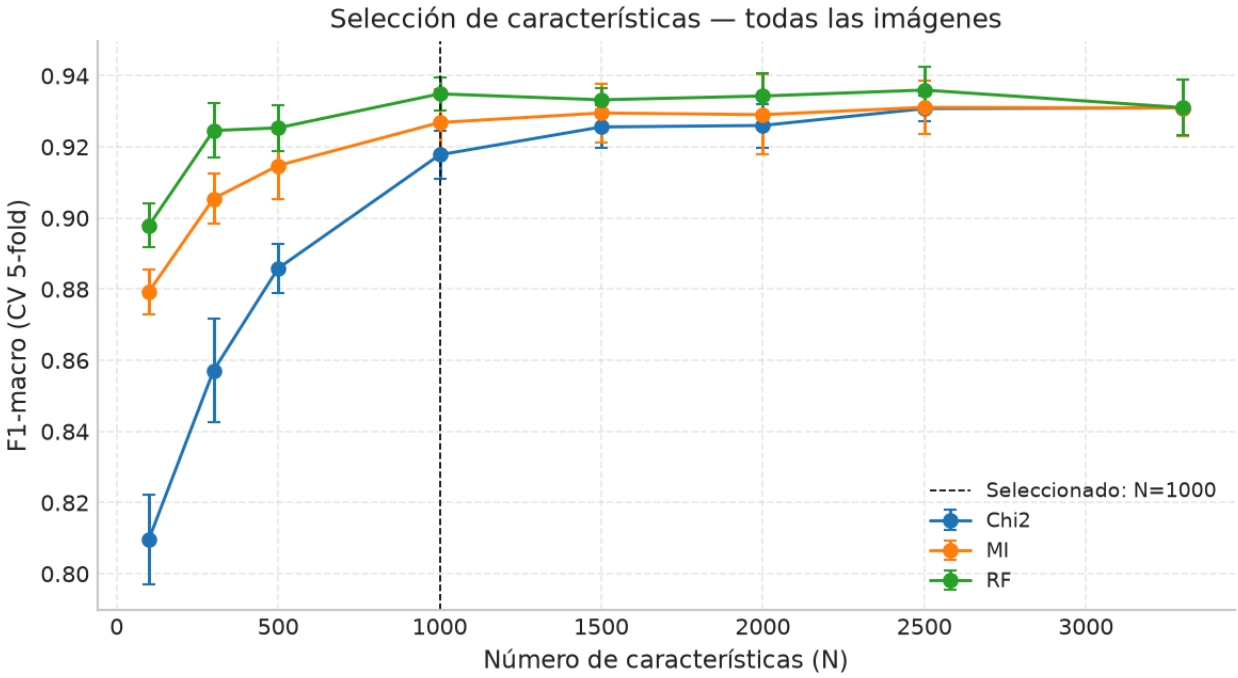
pd.DataFrame({
    'feature': [feat_cols[i] for i in orden_all],
    'score': np.sort(scores_all)[::-1],
}).to_csv(DATA_DIR / '06_scores.csv', index=False)

# Curva F1 vs N.
fig, ax = plt.subplots(figsize=(8, 4.5))
for method in ['Chi2', 'MI', 'RF']:
    sub = resumen[resumen['method'] == method].sort_values('n')
    ax.errorbar(sub['n'], sub['mean'], yerr=sub['std'], marker='o',
                capsize=3, lw=1.4, label=method)
ax.axvline(N_SEL, color='k', linestyle='--', lw=0.8,
           label=f'Seleccionado: N={N_SEL}')
ax.set_xlabel('Número de características (N)', fontsize=11)
ax.set_ylabel('F1-macro (CV 5-fold)', fontsize=11)
ax.set_title('Selección de características — todas las imágenes', fontsize=12)
ax.legend(fontsize=9)
ax.grid(True, linestyle='--', alpha=0.4)
fig.tight_layout()
plt.show()
```

```
Estabilidad (Jaccard medio top-1000 entre folds): 0.4965
Regla de 1 error estándar:
  Mejor configuración : RF, N=2500 (F1=0.935983)
  Error estándar      : 0.002896
  Umbral              : 0.933087

Configuraciones candidatas: 4
method    n     mean      std       se
    RF 1000 0.934914 0.004552 0.002036
    RF 1500 0.933234 0.003388 0.001515
    RF 2000 0.934309 0.006296 0.002816
    RF 2500 0.935983 0.006475 0.002896

Seleccionada (menor N dentro de 1 SE): RF, N=1000
  F1-macro CV : 0.934914 +/- 0.004552
```



Conclusiones

Se usaron **todas las imágenes** (8274 tras deduplicar). No hay hold-out ni conjunto de test separado: la estimación se obtiene por **validación cruzada 5-fold sobre las 8274**.

En cada fold, el selector de características se ajustó **solo con el fold de entrenamiento**. Se compararon Chi^2, Mutual Information e importancia de Random Forest sobre un rango de N, y se eligió el menor N dentro de **1 error estándar** del mejor F1-macro.

Los artefactos `X_full.npy` y `y_full.npy` contienen las 8274 imágenes; el notebook 07 usará exactamente esos datos para la selección de la estrategia de desbalanceo.

Limitación documentada: `N` se elige por CV sobre las 8274 y se reutiliza en el notebook 08; no se retunea dentro de cada fold del 08 por coste computacional. La estabilidad entre folds se reporta con el Jaccard del top-N.

Se verificaron duplicados exactos por hash de archivo y duplicados perceptuales exactos con dHash. dHash detecta imágenes idénticas o casi idénticas, pero no recortes o rotaciones; esa comprobación queda como limitación.

Siguiente etapa: `07_Desbalanceo.ipynb`